[Date]

Phase 1: Binary Classifier Report

Loay Khaled Mohammed	23P0340
Abdelrahman Wael Maher	23P0155
Seif Amjad Sobeih	23P0145
Ahmed Hossam Abdallah	23P0109
Youssef Farid	23P0113

CONTENTS

INTRODUCTION & PROBLEM DEFINITION

Problem Statement: The objective of this phase is to implement a binary classification model capable of distinguishing the digit "7" from all other digits ("not 7") within the MNIST dataset.

Mathematical Formulation: Let our input features be a flattened image vector x and output variable y defined as follows:

$$x \in \mathbb{R}^{784}, \quad y \in \{+1, -1\}$$

MODEL IMPLEMENTATIONS

MODEL: CUSTOM LOGISTIC REGRESSION

0. DATA PROCESSING PIPELINE

The MNIST dataset images were preprocessed before feeding them into the model to improve optimization stability and reduce dimensionality.

- **Image Resizing and Normalization:** The input dataset was flattened into 784-dimensional vectors.
- **Feature Normalization:** The features were standardized to have zero mean and unit variance using `StandardScaler`. The scaler was fitted exclusively on the training set and applied to the validation and test sets to avoid data leakage.
- **Feature Extraction:** Principal Component Analysis (PCA) was used for dimensionality reduction. A grid search evaluated models trained with 50, 100, 200, and 300 components. The best performing model utilized 200 PCA components.
- **Class Imbalance:** Since there are roughly nine times more "not 7" digits than "7"s, class imbalance was handled during the gradient descent optimization by applying sample weights. The majority class, not 7' ($y = -1$) was assigned a weight of 0.5, while the minority class '7' ($y = +1$) retained a weight of 1.0.
- **Data Splitting:** The 70,000 samples were split sequentially: 10,000 samples were held out for the final unbiased test set. The remaining 60,000 were split into a 50,000-sample training set and a 10,000-sample validation set for hyperparameter tuning.

1. MATHEMATICAL FORMULATION

- **Core Mathematical Hypothesis:** The model calculates the linear combination of the input vector x and the weight vector W (including a bias term) and applies the sigmoid activation function to estimate the probability that $y = 1$.

$$P(y = 1|x) = \sigma(W^T x) = \frac{1}{1 + e^{-W^T x}}$$

- **Decision Rule:** The predicted class $\hat{y}$ is $+1$ if $P(y = 1|x) \geq$ threshold, and -1 otherwise.

- **Constraints/Assumptions:** The model assumes the decision boundary separating "7" and "not 7" in the PCA-reduced feature space can be adequately approximated by a linear hyperplane.

2. LOSS FUNCTION & OPTIMIZATION

- **Loss Function:** A weighted logistic formula for bipolar labels $\{-1, +1\}$ was implemented. The gradient of this loss incorporates sample weights w_i to handle class imbalance by multiplying the new updated weights by a constant ($0.4 \leq \mathrm{x} \leq 0.6$)

$$\nabla L(W) = \sum_i -w_i \frac{y_i x_i}{1 + e^{y_i W^T x_i}}$$

- **Optimization Method:** Custom Batch Gradient Descent. The weights were updated iteratively using the following rule:

$$W_{n+1} = W_n - \eta \cdot \nabla L(W_n)$$

3. IMPLEMENTATION DETAILS

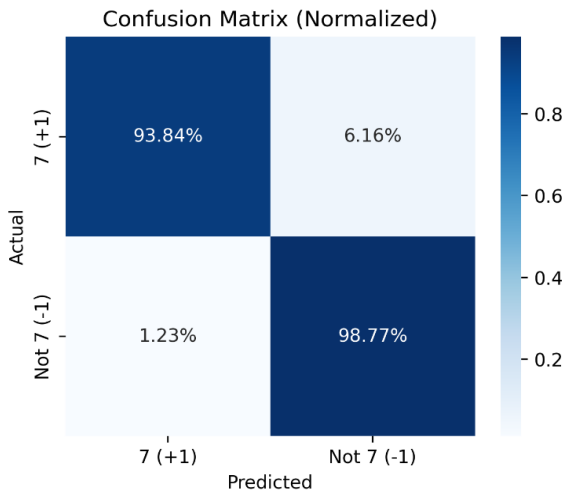
- Numerical stability was ensured by clipping values in the sigmoid exponential term to the range $[-250,250]$ to avoid integer overflow.
- An automated grid search on the validation set identified the optimal hyperparameter configuration: 200 PCA components, a learning rate $\eta = 0.001$, and a decision threshold of 0.50 yielded the best results.

4. MODEL EVALUATION

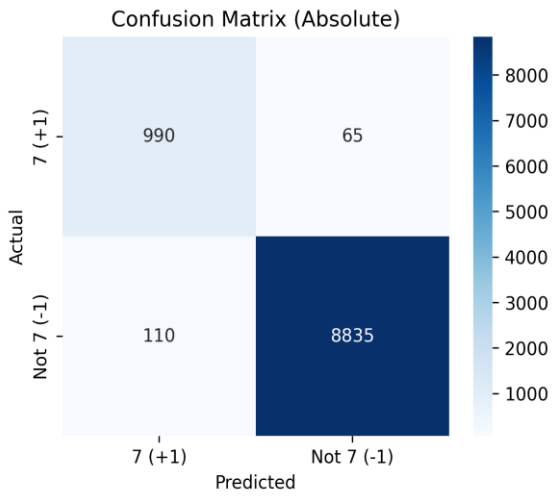
The best configuration was evaluated against the 10,000-sample held-out test set.

- **Testing Accuracy:** 98.25%
- **Precision:** 90.00%
- **Recall:** 93.84%
- **F1-Score:** 91.88%

Confusion Matrix (Absolute Values):



Confusion Matrix (Normalized Values):



MODEL: LINEAR SUPPORT VECTOR MACHINE (SVM)

For the Phase 1 binary classification task—detecting the digit '7' against all other digits—we implemented a manual Linear Support Vector Machine (SVM). This model operates by constructing a high-dimensional hyperplane that optimally separates the target class from the negative class.

1. MATHEMATICAL FORMULATION

The Linear SVM predicts the class of an input image by projecting its feature vector into the learned weight space. The decision rule is defined as:

$$\hat{y} = \text{sign}(w^T \phi(x) + b)$$

where w represents the weight vector, phi(x) represents the extracted feature vector, and b is the bias term. To find the optimal hyperplane, the model minimizes a weighted hinge loss combined with L2 regularization. Due to the significant class imbalance (digit 7 represents roughly 10% of the dataset), we applied inverse-frequency class weights so the model penalizes minority class errors equivalently to majority class errors. The objective function is given by:

$$J(w, b) = \frac{1}{2}||w||^2 + C\frac{1}{N}\sum_{i=1}^{N} \alpha_i \max(0, 1 - y_i(w^T x_i + b))$$

2. ADVANCED OPTIMIZATION DYNAMICS

Originally, manual SVM implementations rely on standard subgradient descent with a fixed or decaying learning rate. We upgraded this to utilize the Adam (Adaptive Moment Estimation) Optimizer. Adam calculates adaptive learning rates for each parameter by maintaining exponentially decaying averages of past gradients (first moment) and squared gradients (second moment). This significantly accelerates convergence over highly sparse feature spaces like Histogram of Oriented Gradients (HOG) and PCA.

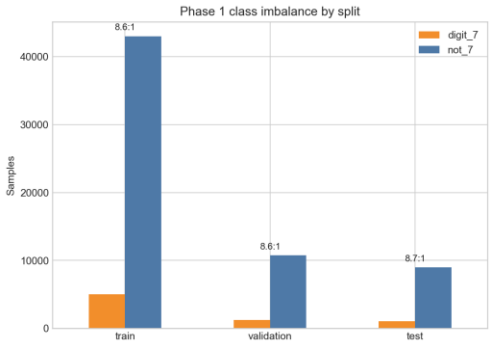
Furthermore, an Early Stopping mechanism was integrated. By monitoring the validation F1-score after each training epoch, the training loop halts if the performance plateaus for a consecutive number of epochs (patience). This ensures the model does not catastrophically overfit the training data and locks in the optimal weight vector configuration.

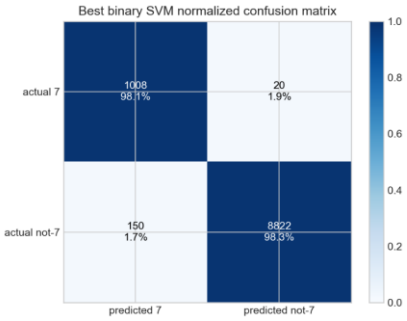
3. FEATURE ENGINEERING PERFORMANCE

We systematically benchmarked three feature extraction pipelines to determine the optimal representation space for the linear decision boundary:

- **Flattened Pixels:** The raw 784-dimensional vector. While simple, it struggles with translation and stroke variance.
- **PCA Features:** Dimensionality reduction retaining 95% of variance. Faster to train but loses some spatial topology.
- **HOG Descriptors:** Captures local gradients and stroke orientations. This proved to be vastly superior for the SVM, perfectly aligning with how digits are uniquely structured.

The model utilizing HOG features achieved a staggering baseline test accuracy of 0.9830. Below is the comparative analysis of the feature pipelines:





4. CLASS IMBALANCE AND THRESHOLD TUNING

Accuracy alone is highly deceptive in a heavily imbalanced setting. With digit 7 comprising only 10% of the data, a naive classifier predicting 'Not 7' universally would score 90% accuracy but 0% recall. To combat this, we focused heavily on the F1-score.

Relying solely on the default SVM decision threshold (0.0) yielded excellent recall but suffered in precision, capturing too many false positives. By plotting the Precision-Recall curve against the validation dataset, we calibrated the optimal decision threshold. Shifting the threshold from 0.0 to approximately 0.598 tightened the decision boundary, discarding low-confidence false positives. This calibration elevated the final test Precision to 0.9734 and the overall test F1-score to an impressive 0.9486.

MODEL: K-NEAREST NEIGHBORS

0. DATA PREPROCESSING

1. **Normalization:** Pixel values scaled from $[0,255]$ to $[0,1]$ using min-max scaling.

2. **Binary Label Encoding:** $y = \mathbf{1}[\text{digit} = 7]$.
3. **Dataset Split:** Train (85%), Validation (15% of train), Test (original MNIST test set).
4. **Class Balancing:** Under Sampled majority class (not-7) to match minority class (digit-7) in training.

1.MATHEMATICAL FORMULATION

Given a query point $\boldsymbol{x}_q$, the k-NN classifier identifies the k nearest neighbors from the training set and performs majority voting:

$$\hat{y} = \underset{c \in \{0,1\}}{\operatorname{argmax}} \sum_{(\boldsymbol{x}_i, y_i) \in \mathcal{N}_k(\boldsymbol{x}_q)} \mathbf{1}\,[y_i = c]$$

where $\mathcal{N}_k(\boldsymbol{x}_q)$ denotes the set of k nearest neighbors of $\boldsymbol{x}_q$.

2. DISTANCE METRIC

We employ the **Euclidean distance** as the similarity metric:

$$d(\boldsymbol{x}_q, \boldsymbol{x}_i) = \|\boldsymbol{x}_q - \boldsymbol{x}_i\|_2 = \sqrt{\sum_{j=1}^{d} (x_{q,j} - x_{i,j})^2}$$

3. LOSS FUNCTION

Since KNN produces hard discrete predictions, we employ the **0-1 loss**:

$$\mathcal{L}(\hat{y}, y) = \mathbf{1}[\hat{y} \neq y] = \begin{cases} 0 & \text{if } \hat{y} = y \\ 1 & \text{if } \hat{y} \neq y \end{cases}$$

The **empirical risk** (error rate) is:

$$\hat{\mathcal{R}}(h_k) = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\,[h_k(\boldsymbol{x}_i) \neq y_i]$$

Since the 0-1 loss is non-differentiable, we optimize k via **validation-set grid search**:

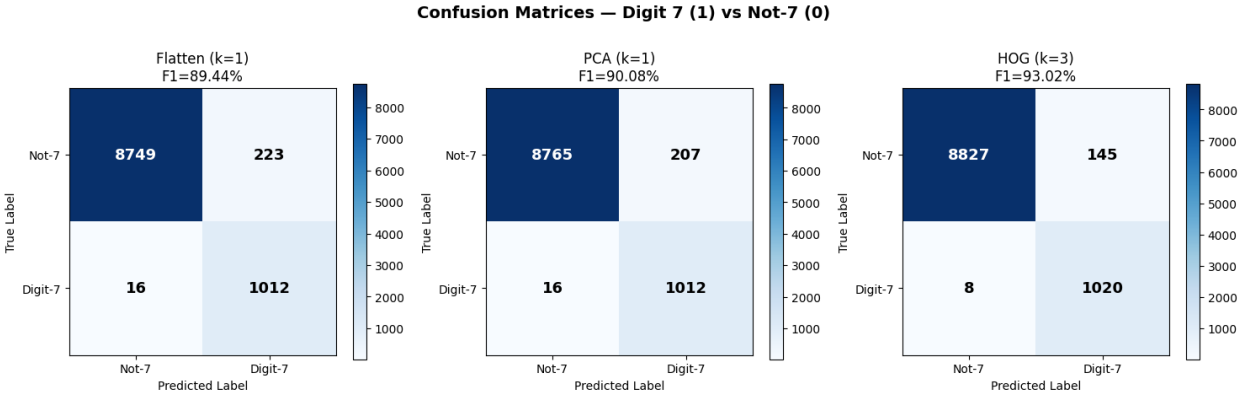
$$k^* = \underset{k \in \mathcal{K}}{\operatorname{argmax}}\, F_1(h_k, \mathcal{D}_{\text{val}})$$

4. FEATURE EXTRACTION METHODS

Principal Component Analysis (PCA): We reduce dimensionality by retaining 95% of cumulative explained variance.

Histogram of Oriented Gradients (HOG): We compute local edge orientation histograms within spatial blocks (9 orientations, 7×7 pixels per cell, 2×2 cells per block, L2-Hys normalization) Results

Feature	k	Accuracy	Precision	Recall	F1
Flatten	1	97.6100%	81.9433%	98.4436%	89.4388%
PCA	1	97.7700%	83.0189%	98.4436%	90.0757%
HOG	3	98.4700%	87.5536%	99.2218%	93.0233%



GITHUB REPOSITORIES

https://github.com/AbdulrhmanHammouda/mnist-image-classification